Reusing the strict Pell index bound

DiophantineConsistency seventh review

26 October 2026 Frozen source `7ba7d102822df`

Result. One optional local cleanup replaces an eleven-line detour in the new outer Pell seed construction with one application of an existing repository theorem. The complete `NativeOuterSeedConstruction` module falls from **279 to 269 physical lines**. All public statements, definitions, imports and witnesses are unchanged. **The candidate is uncompiled.**

This is source cleanup, not new Pell theory or a change to the Diophantine equation. All six earlier deliveries and every prior advertised mechanism, optional branch, notes-only direction and real-client adapter are assumed applied. The other investigated shortenings are excluded rather than relabeled as new ideas.

1 Source and exclusion boundaries

The source pin is `7ba7d102822df1eff376cbca913727d9d588ab4e`, committed at 19:25:35 UTC. The complete, untruncated active Lean tree has 724 files. Against the preceding source pin, 22 were added and twelve modified; against its closing pin, 22 were added and seven modified. Neither comparison removes a file. Counts come from complete trees, not capped compare-file arrays.

The eleven new main/check pairs connect controller safety, rank calibration and stride, halt packing, outer Pell seeds and positive source construction. These new results are not simplification savings. Already-adopted edits in eleven older modules save 186 net lines since the preceding source pin; 156 were already present at its closing screen. The remaining adoption delta is six modules and 30 lines. None is credited here.

Closing boundary. A single closing screen is frozen at `3b51c69d42ff99bc420698532af3fe1d1f9bd356`, committed at 19:32:32 UTC. The target file remains byte-identical. The exclusion inventory contains 43 placed packets and seven incoming ZIPs: the sixth delivery, two new manuscripts at the source pin, and four further manuscripts at closing. Their advertised outcomes are excluded irrespective of whether they have been merged. No later main polling was performed.

2 A Use the exact existing inequality

Source anchors. `NativeOuterSeedConstruction.lean` lines 208–218.

Existing helper: `Diophantine.lt_yn_of_two_le`.

The client needs $r + 1 < K$. Its current proof derives the stronger lower bound $2P - 1 \le K$ through a variable power, compares X, Q and P, then finishes by arithmetic. The repository already proves $n < \mathrm{yn}_a(n)$ under $1 < a$ and $2 \le n$. The replacement is simply:

```
have hKr : r+1 < K := lt_yn_of_two_le hP (by omega)
```

Here `K := yn hP (r+1)`, `hP : 1 < P`, and `population_sizes` supplies `hr : 4 ≤ r`. Thus `omega` proves only $2 \le r + 1$; it does not normalize a large power or re-prove a growth theorem. The later doubled-ordinate bound and positive quotient construction are unchanged.

The helper is already imported through the native Pell development and `Diophantine.Common.PellInt/PellMod`. No helper or import is added. The exact patch changes one local fragment, **11 lines to 1**; all other source bytes, including the existing `hRX` binding, remain identical.

3 Tradeoffs and deliberately excluded alternatives

Maintainability. A states exactly what the client needs and removes stronger intermediate estimates. Those estimates remain available in the shared Pell development, but are no longer visible locally. The shorter proof depends explicitly on the existing helper's name and signature. No elaboration-speed, proof-term-size or axiom-footprint improvement is claimed.

Replace the neighboring bound by a fixed Pell anchor.
Deriving `hcX` from the index-two value and index monotonicity repeats the earlier fixed-index-anchor mechanism. Its earlier example used index six; a different numeral is not a new method.
Dualize the new consistency equivalence.
A `not_congr` proof of `no_positive_zero_iff_conZFC` is shorter, but an earlier report expressly reserves systematic replacement of handwritten contraposition branches after an existing equivalence.
Reuse exact sparse lookup to bound a coefficient.
The positive branch of `abs_coeff_combined_le` duplicates its preceding coefficient proof. Reusing it fits the older injective sparse-placement contract, which already promises image lookup and coefficient bounds together.
Refactor controller arithmetic.
Finite sums, weighted point updates, safe subset construction, same-window capacity, affine positions, rotation/rephasing and source-guard recovery are already covered, including their advertised adapters.

A does not introduce the old two-base comparison, fixed-index anchor or normalized exponential-growth mechanisms. It only removes a new client's rederivation of the exact existing strict index bound. The packet retains the nearest prior texts, semantic decisions and reading coverage. Compilation would establish neither novelty nor a new mathematical result.

4 Replay and remaining integration gates

From the extracted packet root:

```
python3 verify_bundle.py --strict
python3 provenance/verify_snapshot.py
python3 baseline_audit/verify_audit.py
python3 research/outer_seed/check_candidate.py
python3 independent_review/check_lemma_reuse.py
bash report/build.sh
```

The checks establish immutable input identities, complete-tree deltas, one exact local replacement, its patch and line counts, and the retained helper's guards and import path. The candidate checker also exercises **1,953** finite Pell recurrence cases and three negative controls: index zero, index one and base one do not satisfy the strict inequality. These are sanity checks, not Lean proofs.

Normal and optimized Python runs retain explicit guards. The sealed release replay checks changed, missing and extra-file failures, and compares every PDF page's extracted text and rendered pixels after an isolated TeX rebuild. The TeX/build and integrity scaffolding is reused from round six. Incoming archive bytes are omitted from the small ZIP; exact intake hash receipts and bounded extraction inventories are retained, with that replay limit stated explicitly.

Required next gate. In an authorized complete checkout at the source pin, compile the replacement module, its existing Checks module and affected reverse dependencies, then compare endpoint axiom reports. Lean and Lake are unavailable here and were not installed. This snapshot is not a complete Lake project; Python and source inspection establish neither elaboration, kernel acceptance, integration nor performance. No upstream file was changed.